MOBILE APPLICATION DEVELOPMENT

Complete Viva & Practical Revision Guide

Java + XML | 196 syllabus topics | practical explanations | SQLite CRUD | examiner follow-ups

Prepared from the uploaded MAD lecture PDF and the 196-topic viva outline. The guide follows the same topic numbering and terminology so it can be used alongside the syllabus.

This guide includes	What you will find
Theory	Simple spoken answers for all 196 listed topics, with purpose and examples.
Practical code	Java/XML patterns for login, registration, storage, Intents, adapters, menus, fragments and more.
Database	SQLiteOpenHelper concepts and INSERT, SELECT, UPDATE and DELETE examples.
Viva practice	Common examiner follow-up questions and concise model answers.

Exam tip: answer in three steps — define the concept, explain why it is used, then connect it to a practical example.

How to use this guide

• First revise the 196 numbered answers in order. They are written in simple sentences that can be spoken during a viva.

• Next practise the Java snippets. Focus on understanding what each method does rather than memorising punctuation alone.

• Then revise the practical walkthroughs and the comparison tables.

• Finally answer the examiner follow-up questions aloud without looking at the answers.

• The uploaded lecture PDF contains visual diagrams and slide explanations. Relevant slide-page references are noted in the guide where useful.

Important scope note: the 196 topics come from the uploaded viva outline. Some implementation details below are explanatory examples to help you understand those topics; follow your lab teacher's prescribed Android Studio template and dependencies where they differ.

Contents

1. Android Fundamentals — Topics 1–12

2. Android Project Structure — Topics 13–21

3. Emulator and Debugging — Topics 22–29

4. Activity and Lifecycle — Topics 30–35

5. UI Components and Layouts — Topics 36–56

6. Input Validation and Events — Topics 57–66

7. Intents — Topics 67–76

8. Preferences and Local Storage — Topics 77–83

9. Adapters and Collections — Topics 84–96

10. Menus — Topics 97–105

11. Fragments and Navigation — Topics 106–116

12. Styles, Themes and Drawables — Topics 117–125

13. Images, Media, Time and Date — Topics 126–133

14. SQLite Database — Topics 134–155

15. Notifications and Permissions — Topics 156–161

16. Java Programming — Topics 162–182

17. XML — Topics 183–188

18. Advanced Android Concepts — Topics 189–196

19. Practical programme explanations | 20. Important differences | 21. SQLite practical | 22. Common viva follow-ups

1. Android Fundamentals

Related lecture slides: pages 2–5 introduce mobile app types, Android history, architecture and Android Studio.

1. Definition

Viva answer: Explain what the concept is.

Explanation / example: Android is open source, so manufacturers and developers can adapt the platform. In an Android lab, we normally build an app that runs on the Android operating system.

2. Purpose

Viva answer: Explain why it is used.

Explanation / example: The layers separate hardware-level work from application-level work. The kernel handles drivers and core services; the runtime executes app code; the framework provides APIs such as Activity and View.

3. Example

Viva answer: Give a small example from your practical program.

Explanation / example: The SDK includes platform APIs, build tools, emulator images and debugging utilities. We select SDK versions in the project configuration.

4. Android Studio

Viva answer: Android Studio is the official Integrated Development Environment (IDE) for Android application development, and it provides a code editor, emulator, debugger and build tools.

Explanation / example: Android Studio is where we create Java classes, XML layouts, run the emulator and inspect errors through Logcat.

5. Android Application Components

Viva answer: The four main Android application components are Activities, Services, Broadcast Receivers and Content Providers.

Explanation / example: An Activity presents a screen, a Service performs work without a screen, a Broadcast Receiver reacts to broadcasts, and a Content Provider exposes structured data.

6. Activity

Viva answer: An Activity represents a single screen of an Android application, such as a login page or registration page.

Explanation / example: For example, the login screen and dashboard can be separate Activities. An Activity is usually declared in AndroidManifest.xml.

7. Service

Viva answer: A Service is an Android component that performs operations in the background without providing a user interface.

Explanation / example: A Service has no normal user interface. Modern Android places restrictions on background work, so long-running work may require a foreground service or WorkManager depending on the task.

8. Broadcast Receiver

Viva answer: A Broadcast Receiver is an Android component that responds to broadcast messages, such as battery-low notifications or system boot events.

Explanation / example: A receiver can respond to system or app broadcasts. For example, a receiver may react to a battery-low broadcast, subject to Android's broadcast restrictions.

9. Content Provider

Viva answer: A Content Provider manages and shares application data with other applications through a standard interface.

Explanation / example: A Content Provider supports controlled data sharing through URIs and CRUD-style operations. Contacts is a familiar example of provider-backed data.

10. Android Runtime (ART)

Viva answer: Android Runtime is the execution environment that runs Android application code and provides services such as memory management and garbage collection.

Explanation / example: ART executes DEX bytecode and manages runtime services such as garbage collection. It replaced Dalvik as Android's runtime.

11. Linux Kernel

Viva answer: The Linux Kernel is the lowest major software layer of Android and manages device drivers, memory, processes, security and hardware communication.

Explanation / example: The Linux Kernel manages low-level resources such as memory, processes, device drivers and security boundaries.

12. APK and AAB

Viva answer: APK is the installable Android application package, whereas AAB is the Android App Bundle format used to publish applications through Google Play, which can generate optimized APKs for devices.

Explanation / example: An APK can be installed directly. An AAB is a publishing format from which Google Play creates device-specific APKs.

> Examiner: What is the difference between Android and Android Studio?
>
> Answer: Android is the operating system; Android Studio is the IDE used to develop Android apps.
>
> Examiner: Why is Android called open source?
>
> Answer: Its source code is available through the Android Open Source Project, subject to its licences.

2. Android Project Structure

13. AndroidManifest.xml

Viva answer: AndroidManifest.xml is an important configuration file that declares application components, permissions, application information and other essential settings.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for androidmanifest.xml and test the behaviour in the emulator.

14. Application Permissions

Viva answer: Permissions allow an Android application to access protected device features such as the camera, contacts, location and microphone.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for application permissions and test the behaviour in the emulator.

15. SDK Versions – minSdk, targetSdk, compileSdk

Viva answer: minSdk specifies the minimum Android version supported, targetSdk indicates the Android version the application is designed and tested for, and compileSdk specifies the Android API version used to compile the application.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for sdk versions – minsdk, targetsdk, compilesdk and test the behaviour in the emulator.

16. Resources and R Class

Viva answer: Android resources include layouts, strings, images and colours, and the automatically generated R class provides identifiers to access these resources from Java code.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for resources and r class and test the behaviour in the emulator.

17. Assets Folder

Viva answer: The assets folder stores application files such as fonts, HTML files and other raw files that can be accessed using the AssetManager.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for assets folder and test the behaviour in the emulator.

18. Layout Resources

Viva answer: Layout resources are XML files that define the user interface structure and position of views in an Android application.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for layout resources and test the behaviour in the emulator.

19. Drawable Resources

Viva answer: Drawable resources contain images and XML-based graphics used for backgrounds, icons, shapes and other visual elements.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for drawable resources and test the behaviour in the emulator.

20. Gradle

Viva answer: Gradle is the build automation system used by Android Studio to compile code, manage dependencies, configure build variants and generate application packages.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for gradle and test the behaviour in the emulator.

21. Application Structure

Viva answer: An Android project generally contains Java or Kotlin source files, XML resources, the manifest file, Gradle configuration files and test directories.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for application structure and test the behaviour in the emulator.

Examiner: What does setImageResource() do?

Answer: It changes the drawable displayed by an ImageView.

Examiner: DatePicker versus TimePicker?

Answer: DatePicker selects a calendar date; TimePicker selects hours and minutes.

3. Emulator and Debugging

22. Android Virtual Device (AVD)

Viva answer: An AVD is a configuration that defines the hardware and Android system image of a virtual device used for testing applications.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for android virtual device (avd) and test the behaviour in the emulator.

23. Android Emulator

Viva answer: The Android Emulator is a software tool that simulates an Android device on a computer so developers can run and test applications without a physical phone.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for android emulator and test the behaviour in the emulator.

24. Emulator Configuration

Viva answer: Emulator configuration includes selecting the device model, screen size, RAM, storage, Android version and system image.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for emulator configuration and test the behaviour in the emulator.

25. Logcat

Viva answer: Logcat is an Android Studio debugging tool that displays application logs, system messages, warnings and errors.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for logcat and test the behaviour in the emulator.

26. DDMS

Viva answer: DDMS stands for Dalvik Debug Monitor Server, which was an older Android debugging tool used for monitoring processes, threads, heap memory and device activities.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for ddms and test the behaviour in the emulator.

27. Debugging

Viva answer: Debugging is the process of finding, analysing and correcting errors in an application using tools such as breakpoints, Logcat and the debugger.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for debugging and test the behaviour in the emulator.

28. Types of Programming Errors

Viva answer: The three common types of programming errors are syntax errors, runtime errors and logical errors.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for types of programming errors and test the behaviour in the emulator.

29. Exception Handling

Viva answer: Exception handling is a mechanism used to handle runtime problems using try, catch and finally so that the application can respond safely.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for exception handling and test the behaviour in the emulator.

4. Activity and Lifecycle

Related lecture slides: pages 12–13 show the Activity lifecycle methods and lifecycle flow diagram.

30. Activity Lifecycle

Viva answer: Activity Lifecycle describes the different states an Activity passes through from creation to destruction.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for activity lifecycle and test the behaviour in the emulator.

31. Lifecycle Methods

Viva answer: The main Activity Lifecycle methods are onCreate(), onStart(), onResume(), onPause(), onStop(), onRestart() and onDestroy().

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for lifecycle methods and test the behaviour in the emulator.

32. Activity Navigation

Viva answer: Activity Navigation is the process of moving from one Activity to another using Intents and the startActivity() method.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for activity navigation and test the behaviour in the emulator.

33. Activity Back Stack

Viva answer: The Activity Back Stack is a last-in, first-out collection that manages the order of Activities and determines which screen appears when the Back button is pressed.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for activity back stack and test the behaviour in the emulator.

34. Context

Viva answer: Context provides access to application resources, system services and application-related information, and it is commonly used when creating Intents, Toast messages and other Android objects.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for context and test the behaviour in the emulator.

35. onCreate() and setContentView()

Viva answer: onCreate() is called when an Activity is created, and setContentView() loads the XML layout that defines its user interface.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for oncreate() and setcontentview() and test the behaviour in the emulator.

5. UI Components and Layouts

Related lecture slides: pages 14–18 explain Views, ViewGroups, layouts, display units and common UI methods.

36. View and ViewGroup

Viva answer: A View is a basic user interface element such as a Button or TextView, whereas a ViewGroup is a container that holds and arranges other Views.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for view and viewgroup and test the behaviour in the emulator.

37. TextView

Viva answer: TextView is a UI component used to display text to the user.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for textview and test the behaviour in the emulator.

38. EditText

Viva answer: EditText is a UI component that allows users to enter and edit text.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for edittext and test the behaviour in the emulator.

39. Button

Viva answer: A Button is a clickable UI component used to perform an action when the user presses it.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for button and test the behaviour in the emulator.

40. ImageView

Viva answer: ImageView is a UI component used to display images from drawable resources or other image sources.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for imageview and test the behaviour in the emulator.

41. CheckBox

Viva answer: A CheckBox allows users to select or deselect an independent option, and multiple CheckBoxes can be selected simultaneously.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for checkbox and test the behaviour in the emulator.

42. RadioButton

Viva answer: A RadioButton is a selection control used when the user needs to choose one option from a group.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for radiobutton and test the behaviour in the emulator.

43. RadioGroup

Viva answer: RadioGroup is a container that groups RadioButtons and normally allows only one RadioButton to be selected at a time.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for radiogroup and test the behaviour in the emulator.

44. Spinner

Viva answer: Spinner is a dropdown UI component that allows users to select one item from a list of available options.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for spinner and test the behaviour in the emulator.

45. ListView

Viva answer: ListView is a UI component that displays a vertically scrollable list of items using an Adapter.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for listview and test the behaviour in the emulator.

46. GridView

Viva answer: GridView displays items in a two-dimensional, scrollable grid consisting of rows and columns.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for gridview and test the behaviour in the emulator.

47. LinearLayout

Viva answer: LinearLayout arranges its child Views either vertically or horizontally using the orientation attribute.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for linearlayout and test the behaviour in the emulator.

48. RelativeLayout

Viva answer: RelativeLayout positions Views relative to the parent layout or relative to other Views.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for relativelayout and test the behaviour in the emulator.

49. FrameLayout

Viva answer: FrameLayout is a simple container that stacks its child Views, often one above another, and is commonly used for fragments and image overlays.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for framelayout and test the behaviour in the emulator.

50. GridLayout

Viva answer: GridLayout arranges Views in a rectangular grid containing rows and columns.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for gridlayout and test the behaviour in the emulator.

51. Nested/Cascaded LinearLayout

Viva answer: Nested LinearLayout means placing one LinearLayout inside another LinearLayout to create complex user interface arrangements.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for nested/cascaded linearlayout and test the behaviour in the emulator.

52. Layout Width and Height

Viva answer: layout_width and layout_height specify the width and height of a View or layout.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for layout width and height and test the behaviour in the emulator.

53. match_parent and wrap_content

Viva answer: match_parent makes a View occupy the available size of its parent, whereas wrap_content makes it occupy only the space required by its content.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for match_parent and wrap_content and test the behaviour in the emulator.

54. Padding and Margin

Viva answer: Padding is the space inside a View between its content and boundary, whereas margin is the space outside a View separating it from other Views.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for padding and margin and test the behaviour in the emulator.

55. Gravity and layout_gravity

Viva answer: gravity controls the position of content inside a View, whereas layout_gravity controls the position of the View inside its parent.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for gravity and layout_gravity and test the behaviour in the emulator.

56. dp, dip, sp and px

Viva answer: dp and dip are density-independent units used for layout dimensions, sp is a scalable unit generally used for text, and px represents actual screen pixels.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for dp, dip, sp and px and test the behaviour in the emulator.

6. Input Validation and Events

57. EditText Validation

Viva answer: EditText validation checks user input before processing or saving it, such as checking whether a username or password field is empty.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for edittext validation and test the behaviour in the emulator.

58. Input Types

Viva answer: The inputType attribute specifies the type of data expected in an EditText, such as text, number, phone number, email or password.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for input types and test the behaviour in the emulator.

59. String Handling

Viva answer: String handling involves creating, reading, comparing and modifying text using Java String methods such as equals(), trim(), isEmpty() and length().

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for string handling and test the behaviour in the emulator.

60. Event Handling

Viva answer: Event handling is the process of responding to user actions such as button clicks, item selections and touch events.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for event handling and test the behaviour in the emulator.

61. Click Listeners

Viva answer: A Click Listener is an event listener that executes code when a user clicks a View.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for click listeners and test the behaviour in the emulator.

62. Lambda Expressions

Viva answer: A Lambda Expression is a shorter way of writing functional interface implementations, such as button click listeners in Java.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for lambda expressions and test the behaviour in the emulator.

63. Toast

Viva answer: Toast is a small popup message that provides temporary feedback to the user without interrupting the application.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for toast and test the behaviour in the emulator.

64. AlertDialog

Viva answer: AlertDialog is a dialog box that displays a title, message and buttons to ask the user for confirmation or provide information.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for alertdialog and test the behaviour in the emulator.

65. Popup

Viva answer: A Popup is a small temporary window that displays additional options or information without opening a new Activity.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for popup and test the behaviour in the emulator.

66. Exception Handling in UI

Viva answer: Exception handling in the user interface prevents invalid operations, such as converting non-numeric input into a number without proper validation.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for exception handling in ui and test the behaviour in the emulator.

7. Intents

Related lecture slides: pages 19 onward introduce Intents and their use.

67. Intent

Viva answer: An Intent is a messaging object used to request an action from another Android component, such as opening an Activity or launching the phone dialer.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for intent and test the behaviour in the emulator.

68. Explicit Intent

Viva answer: An Explicit Intent specifies the exact Activity or component to open, such as navigating from MainActivity to MainActivity2.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for explicit intent and test the behaviour in the emulator.

69. Implicit Intent

Viva answer: An Implicit Intent specifies an action to perform without naming the exact application component, such as opening a browser or sharing text.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for implicit intent and test the behaviour in the emulator.

70. Intent Filters

Viva answer: Intent Filters are manifest declarations that specify the types of implicit Intent actions, categories and data an Activity or other component can handle.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for intent filters and test the behaviour in the emulator.

71. Intent Actions

Viva answer: Intent Actions describe the operation requested by an Intent, such as ACTION_VIEW, ACTION_SEND and ACTION_DIAL.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for intent actions and test the behaviour in the emulator.

72. Passing Data Between Activities

Viva answer: Data can be passed between Activities by adding values to an Intent using putExtra() and retrieving them in the destination Activity.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for passing data between activities and test the behaviour in the emulator.

73. putExtra() and getStringExtra()

Viva answer: putExtra() attaches data to an Intent, whereas getStringExtra() retrieves a String value from the Intent.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for putextra() and getstringextra() and test the behaviour in the emulator.

74. startActivity()

Viva answer: startActivity() launches another Activity using the supplied Intent.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for startactivity() and test the behaviour in the emulator.

75. finish()

Viva answer: finish() closes the current Activity and removes it from the Activity task's active stack.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for finish() and test the behaviour in the emulator.

76. Activity Navigation

Viva answer: Activity Navigation connects different application screens using Intents and manages how users move between them.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for activity navigation and test the behaviour in the emulator.

8. Preferences and Local Storage

77. SharedPreferences

Viva answer: SharedPreferences is a lightweight local storage mechanism used to store small amounts of data as key-value pairs.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for sharedpreferences and test the behaviour in the emulator.

78. SharedPreferences.Editor

Viva answer: SharedPreferences.Editor is used to add, modify or remove values in a SharedPreferences file.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for sharedpreferences.editor and test the behaviour in the emulator.

79. apply() and commit()

Viva answer: apply() saves changes asynchronously without returning a result, whereas commit() saves changes synchronously and returns a Boolean indicating success.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for apply() and commit() and test the behaviour in the emulator.

80. putString() and getString()

Viva answer: putString() stores a String value using a key, whereas getString() retrieves the stored String using that key and a default value.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for putstring() and getstring() and test the behaviour in the emulator.

81. Storing Registration Details

Viva answer: Registration details such as username, email and phone number can be stored locally using SharedPreferences or in a structured SQLite database.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for storing registration details and test the behaviour in the emulator.

82. SharedPreferences vs SQLite

Viva answer: SharedPreferences is suitable for small key-value data such as settings and login status, whereas SQLite is suitable for structured data stored in tables with rows and columns.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for sharedpreferences vs sqlite and test the behaviour in the emulator.

83. Secure Storage Basics

Viva answer: Secure storage means protecting sensitive information using appropriate security mechanisms, avoiding plain-text password storage and applying encryption or secure platform storage where necessary.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for secure storage basics and test the behaviour in the emulator.

9. Adapters and Collections

84. Adapter

Viva answer: An Adapter acts as a bridge between a data source and a UI component such as ListView, GridView or Spinner.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for adapter and test the behaviour in the emulator.

85. ArrayAdapter

Viva answer: ArrayAdapter connects an array or list of objects to an AdapterView and converts each item into a View for display.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for arrayadapter and test the behaviour in the emulator.

86. BaseAdapter

Viva answer: BaseAdapter is a base class used to create custom adapters when we need more control over how list or grid items are displayed.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for baseadapter and test the behaviour in the emulator.

87. SimpleAdapter

Viva answer: SimpleAdapter maps data from a list of Maps, such as key-value pairs, to Views defined in an XML layout.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for simpleadapter and test the behaviour in the emulator.

88. CursorAdapter

Viva answer: CursorAdapter connects data retrieved through a database Cursor to UI components such as ListView.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for cursoradapter and test the behaviour in the emulator.

89. ListView with ArrayAdapter

Viva answer: ListView with ArrayAdapter displays items from an array or ArrayList as a vertically scrollable list.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for listview with arrayadapter and test the behaviour in the emulator.

90. GridView with Adapter

Viva answer: GridView with an Adapter displays data items in a grid, and the Adapter supplies the content for each grid cell.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for gridview with adapter and test the behaviour in the emulator.

91. Spinner with Adapter

Viva answer: Spinner uses an Adapter to display a collection of choices in a dropdown list.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for spinner with adapter and test the behaviour in the emulator.

92. setAdapter()

Viva answer: setAdapter() attaches an Adapter to a UI component so that it can display data supplied by the Adapter.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for setadapter() and test the behaviour in the emulator.

93. Item Click Listener

Viva answer: An Item Click Listener responds when a user selects an item in components such as ListView or GridView.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for item click listener and test the behaviour in the emulator.

94. simple_list_item_1

Viva answer: simple_list_item_1 is a built-in Android layout containing one TextView that can be used to display a single line of text in a list.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for simple_list_item_1 and test the behaviour in the emulator.

95. simple_list_item_2

Viva answer: simple_list_item_2 is a built-in Android layout containing two TextViews, commonly used to display a primary line and a secondary line of text.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for simple_list_item_2 and test the behaviour in the emulator.

96. Exception Handling in Adapters

Viva answer: Exception handling in Adapters prevents application crashes caused by invalid data, missing values or incorrect positions when displaying collection items.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for exception handling in adapters and test the behaviour in the emulator.

10. Menus

97. Options Menu

Viva answer: Options Menu displays actions related to the current Activity, usually through the three-dot overflow menu in the app bar.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for options menu and test the behaviour in the emulator.

98. Context Menu

Viva answer: Context Menu displays actions related to a particular View or selected item, usually when the user performs a long press.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for context menu and test the behaviour in the emulator.

99. Popup Menu

Viva answer: Popup Menu displays a small menu anchored to a particular View, such as a button.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for popup menu and test the behaviour in the emulator.

100. Menu XML

Viva answer: Menu XML is a resource file used to define menu items, their IDs, titles, icons and other menu properties.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for menu xml and test the behaviour in the emulator.

101. MenuInflater

Viva answer: MenuInflater converts an XML menu resource into actual Menu objects that can be displayed by an Activity.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for menuinflater and test the behaviour in the emulator.

102. onCreateOptionsMenu()

Viva answer: onCreateOptionsMenu() is a callback method used to create and display the Options Menu of an Activity.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for oncreateoptionsmenu() and test the behaviour in the emulator.

103. onOptionsItemSelected()

Viva answer: onOptionsItemSelected() is called when the user selects an item from the Options Menu.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for onoptionsitemselected() and test the behaviour in the emulator.

104. onCreateContextMenu()

Viva answer: onCreateContextMenu() is a callback method used to create a Context Menu for a View.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for oncreatecontextmenu() and test the behaviour in the emulator.

105. onContextItemSelected()

Viva answer: onContextItemSelected() handles the user's selection from a Context Menu.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for oncontextitemselected() and test the behaviour in the emulator.

11. Fragments and Navigation

106. Fragment

Viva answer: A Fragment is a reusable portion of an Activity's user interface and behaviour that can be added, removed or replaced dynamically.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for fragment and test the behaviour in the emulator.

107. Fragment Lifecycle

Viva answer: Fragment Lifecycle describes the stages of a Fragment, including creation, view creation, becoming visible, interaction and destruction.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for fragment lifecycle and test the behaviour in the emulator.

108. FragmentManager

Viva answer: FragmentManager manages Fragments inside an Activity, including adding, removing, finding and replacing them.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for fragmentmanager and test the behaviour in the emulator.

109. FragmentTransaction

Viva answer: FragmentTransaction is used to perform operations such as adding, removing, replacing or showing Fragments.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for fragmenttransaction and test the behaviour in the emulator.

110. Add and Replace Fragments

Viva answer: The add() operation places a Fragment into a container, whereas replace() removes the existing Fragment from that container and places another Fragment there.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for add and replace fragments and test the behaviour in the emulator.

111. Back Stack

Viva answer: The Fragment Back Stack stores Fragment transactions so that the user can return to a previous Fragment state using the Back button.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for back stack and test the behaviour in the emulator.

112. Navigation Drawer

Viva answer: Navigation Drawer is a side navigation panel that provides links to different sections of an application.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for navigation drawer and test the behaviour in the emulator.

113. DrawerLayout

Viva answer: DrawerLayout is a layout container that manages the main screen and a sliding navigation drawer.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for drawerlayout and test the behaviour in the emulator.

114. NavigationView

Viva answer: NavigationView is a UI component commonly used inside DrawerLayout to display navigation menu items.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for navigationview and test the behaviour in the emulator.

115. Tabs

Viva answer: Tabs divide an application interface into multiple sections, allowing users to switch between related screens.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for tabs and test the behaviour in the emulator.

116. TabLayout

Viva answer: TabLayout is an Android UI component used to display and manage tabs, often in combination with ViewPager2.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for tablayout and test the behaviour in the emulator.

12. Styles, Themes and Drawables

117. Style

Viva answer: A Style is a collection of visual properties that can be applied to an individual View, such as text size, text colour and background.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for style and test the behaviour in the emulator.

118. Theme

Viva answer: A Theme is a collection of visual properties applied to an entire application, Activity or view hierarchy.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for theme and test the behaviour in the emulator.

119. Style vs Theme

Viva answer: A Style generally controls the appearance of individual Views, whereas a Theme controls the overall appearance of an application or Activity.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for style vs theme and test the behaviour in the emulator.

120. styles.xml / themes.xml

Viva answer: These resource files define reusable styles and themes that can be applied to application components.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for styles.xml / themes.xml and test the behaviour in the emulator.

121. Applying Themes Through XML

Viva answer: A Theme can be applied through the android:theme attribute in the application or Activity declaration in AndroidManifest.xml.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for applying themes through xml and test the behaviour in the emulator.

122. Applying Themes Through Java

Viva answer: A Theme can be applied programmatically using methods such as setTheme() before setting the Activity's content view.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for applying themes through java and test the behaviour in the emulator.

123. Shape Drawable

Viva answer: Shape Drawable is an XML drawable used to create shapes such as rectangles, circles, rounded corners and borders without requiring image files.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for shape drawable and test the behaviour in the emulator.

124. Gradient Drawable

Viva answer: Gradient Drawable is a drawable that creates a gradual transition between two or more colours.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for gradient drawable and test the behaviour in the emulator.

125. Selector / State List Drawable

Viva answer: A Selector or State List Drawable defines different drawable appearances for different View states, such as pressed, focused, selected or normal.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for selector / state list drawable and test the behaviour in the emulator.

13. Images, Media, Time and Date

126. ImageView

Viva answer: ImageView is a UI component used to display images in an Android application.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for imageview and test the behaviour in the emulator.

127. Image Resources

Viva answer: Image Resources are image files stored in resource directories such as res/drawable and accessed through the generated R class.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for image resources and test the behaviour in the emulator.

128. Image Toggle

Viva answer: Image Toggle means changing an image between two or more states, such as switching between an on image and an off image when a button is clicked.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for image toggle and test the behaviour in the emulator.

129. setImageResource()

Viva answer: setImageResource() changes the image displayed in an ImageView by assigning a drawable resource ID.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for setimageresource() and test the behaviour in the emulator.

130. DatePicker

Viva answer: DatePicker is a UI component that allows the user to select a date consisting of a day, month and year.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for datepicker and test the behaviour in the emulator.

131. TimePicker

Viva answer: TimePicker is a UI component that allows the user to select a time in hours and minutes.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for timepicker and test the behaviour in the emulator.

132. MediaPlayer

Viva answer: MediaPlayer is an Android class used to play audio and video media from supported sources.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for mediaplayer and test the behaviour in the emulator.

133. Drawable vs Mipmap

Viva answer: Drawable resources are commonly used for images and graphics displayed inside the application, whereas mipmap resources are primarily used for launcher icons at different resolutions.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for drawable vs mipmap and test the behaviour in the emulator.

14. SQLite Database

134. SQLite

Viva answer: SQLite is a lightweight, embedded relational database that stores structured data locally on an Android device without requiring a separate database server.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for sqlite and test the behaviour in the emulator.

135. Relational Database

Viva answer: A Relational Database stores information in tables consisting of rows and columns, and relationships can be established between tables using keys.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for relational database and test the behaviour in the emulator.

136. Database, Table, Row and Column

Viva answer: A Database is a collection of related tables, a Table stores related information, a Row represents one record, and a Column represents one attribute of that record.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for database, table, row and column and test the behaviour in the emulator.

137. Primary Key

Viva answer: A Primary Key is a column or combination of columns that uniquely identifies every record in a database table.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for primary key and test the behaviour in the emulator.

138. SQLiteOpenHelper

Viva answer: SQLiteOpenHelper is an Android helper class used to create, open, manage and upgrade an SQLite database.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for sqliteopenhelper and test the behaviour in the emulator.

139. onCreate() and onUpgrade()

Viva answer: onCreate() is called when the database is created for the first time, whereas onUpgrade() is called when the database version is increased.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for oncreate() and onupgrade() and test the behaviour in the emulator.

140. getWritableDatabase()

Viva answer: getWritableDatabase() opens or creates an SQLite database for reading and writing operations.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for getwritabledatabase() and test the behaviour in the emulator.

141. getReadableDatabase()

Viva answer: getReadableDatabase() opens a database for reading and may provide a read-only database if writing is not possible.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for getreadabledatabase() and test the behaviour in the emulator.

142. ContentValues

Viva answer: ContentValues is an Android class that stores database column names and their corresponding values in key-value pairs, especially for INSERT and UPDATE operations.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for contentvalues and test the behaviour in the emulator.

143. Cursor

Viva answer: Cursor is an object that provides access to the rows returned by a database query and allows us to read the retrieved records.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for cursor and test the behaviour in the emulator.

144. moveToFirst()

Viva answer: moveToFirst() moves the Cursor to the first row of the query result and returns false if no rows exist.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for movetofirst() and test the behaviour in the emulator.

145. moveToNext()

Viva answer: moveToNext() moves the Cursor to the next row and returns false when there are no more rows.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for movetonext() and test the behaviour in the emulator.

146. CRUD Operations

Viva answer: CRUD stands for Create, Read, Update and Delete, which are the four basic operations used to manage database records.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for crud operations and test the behaviour in the emulator.

147. INSERT

Viva answer: INSERT is an SQL command used to add a new record into a database table.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for insert and test the behaviour in the emulator.

148. SELECT

Viva answer: SELECT is an SQL command used to retrieve records from a database table.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for select and test the behaviour in the emulator.

149. UPDATE

Viva answer: UPDATE is an SQL command used to modify existing records in a database table.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for update and test the behaviour in the emulator.

150. DELETE

Viva answer: DELETE is an SQL command used to remove one or more records from a database table.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for delete and test the behaviour in the emulator.

151. WHERE Clause

Viva answer: WHERE is an SQL clause used to filter records based on a specified condition during SELECT, UPDATE or DELETE operations.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for where clause and test the behaviour in the emulator.

152. DELETE vs DROP

Viva answer: DELETE removes selected or all records from a table while keeping the table structure, whereas DROP removes the entire table structure along with its records.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for delete vs drop and test the behaviour in the emulator.

153. SQLite vs SharedPreferences

Viva answer: SQLite is used for structured data involving tables and multiple records, whereas SharedPreferences is used for small key-value information such as settings and login status.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for sqlite vs sharedpreferences and test the behaviour in the emulator.

154. SQL Injection

Viva answer: SQL Injection is a security vulnerability in which malicious SQL input changes the intended database query, and it can be prevented by using parameterized queries and selection arguments.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for sql injection and test the behaviour in the emulator.

155. Database Closing and Cursor Closing

Viva answer: A Cursor should be closed after reading query results, and database resources should be closed when they are no longer required to avoid resource leaks.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for database closing and cursor closing and test the behaviour in the emulator.

15. Notifications and Permissions

156. Android Notifications

Viva answer: Android Notifications are messages displayed outside the application's user interface to inform users about events, reminders or updates.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for android notifications and test the behaviour in the emulator.

157. Notification Channel

Viva answer: A Notification Channel is a category introduced in Android 8.0 that allows users to manage the importance and behaviour of different types of notifications.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for notification channel and test the behaviour in the emulator.

158. Runtime Permissions

Viva answer: Runtime Permissions are permissions that an application must request from the user while the application is running, such as access to location or camera.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for runtime permissions and test the behaviour in the emulator.

159. Normal Permissions

Viva answer: Normal Permissions allow access to features that involve minimal risk to user privacy and are generally granted automatically by the Android system.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for normal permissions and test the behaviour in the emulator.

160. Dangerous Permissions

Viva answer: Dangerous Permissions provide access to sensitive user data or device features and generally require explicit user approval at runtime.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for dangerous permissions and test the behaviour in the emulator.

161. Application Sandbox

Viva answer: Android Application Sandbox isolates each application from other applications using separate user IDs and security restrictions to protect application data.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for application sandbox and test the behaviour in the emulator.

16. Java Programming

162. Java Fundamentals

Viva answer: Java is an object-oriented, class-based programming language used in Android development to implement application logic.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for java fundamentals and test the behaviour in the emulator.

163. Classes and Objects

Viva answer: A Class is a blueprint that defines variables and methods, whereas an Object is an instance of a class.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for classes and objects and test the behaviour in the emulator.

164. Variables and Data Types

Viva answer: A Variable is a named memory location used to store a value, and a Data Type specifies the kind of value that the variable can hold.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for variables and data types and test the behaviour in the emulator.

165. Methods

Viva answer: A Method is a block of code that performs a particular task and can be called whenever required.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for methods and test the behaviour in the emulator.

166. Constructors

Viva answer: A Constructor is a special class member used to initialize objects, and it has the same name as the class.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for constructors and test the behaviour in the emulator.

167. Inheritance

Viva answer: Inheritance allows one class to acquire the properties and methods of another class using the extends keyword.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for inheritance and test the behaviour in the emulator.

168. Polymorphism

Viva answer: Polymorphism means one interface or method name can represent different behaviours depending on the object or arguments involved.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for polymorphism and test the behaviour in the emulator.

169. Encapsulation

Viva answer: Encapsulation means combining data and methods inside a class and controlling access to data using access modifiers.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for encapsulation and test the behaviour in the emulator.

170. Abstraction

Viva answer: Abstraction means hiding implementation details and exposing only the essential features of an object.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for abstraction and test the behaviour in the emulator.

171. Interfaces

Viva answer: An Interface defines a contract containing methods that implementing classes must provide, subject to Java's interface rules.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for interfaces and test the behaviour in the emulator.

172. Arrays

Viva answer: An Array is a fixed-size collection of elements of the same data type stored under one variable name.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for arrays and test the behaviour in the emulator.

173. Strings

Viva answer: A String is an object representing a sequence of characters, and Java Strings are immutable.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for strings and test the behaviour in the emulator.

174. static and final

Viva answer: static makes a member belong to the class rather than an individual object, whereas final prevents a variable from being reassigned, a method from being overridden or a class from being inherited, depending on its use.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for static and final and test the behaviour in the emulator.

175. this and super

Viva answer: this refers to the current object, whereas super refers to the immediate parent class.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for this and super and test the behaviour in the emulator.

176. Method Overloading

Viva answer: Method Overloading means defining multiple methods with the same name but different parameter lists in the same class.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for method overloading and test the behaviour in the emulator.

177. Method Overriding

Viva answer: Method Overriding occurs when a subclass provides its own implementation of an inherited method with a compatible method signature.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for method overriding and test the behaviour in the emulator.

178. Lambda Expressions

Viva answer: Lambda Expressions provide a concise way to implement functional interfaces and are commonly used for event handling in Android.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for lambda expressions and test the behaviour in the emulator.

179. try, catch and finally

Viva answer: try contains code that may cause an exception, catch handles the exception, and finally executes cleanup code whether an exception occurs or not, subject to normal control-flow termination.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for try, catch and finally and test the behaviour in the emulator.

180. NullPointerException

Viva answer: NullPointerException occurs when a program tries to access a method or field through a reference that contains null.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for nullpointerexception and test the behaviour in the emulator.

181. NumberFormatException

Viva answer: NumberFormatException occurs when a String cannot be converted into the requested numeric data type.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for numberformatexception and test the behaviour in the emulator.

182. Compile-time and Runtime Errors

Viva answer: Compile-time errors are detected during compilation, whereas runtime errors occur while the program is executing.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for compile-time and runtime errors and test the behaviour in the emulator.

17. XML

183. Introduction to XML

Viva answer: XML stands for Extensible Markup Language, and Android uses XML to define layouts, strings, colours, menus and other application resources.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for introduction to xml and test the behaviour in the emulator.

184. XML Elements and Attributes

Viva answer: XML Elements define the components or structure of a document, whereas Attributes provide additional properties or information about those elements.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for xml elements and attributes and test the behaviour in the emulator.

185. XML Layout Structure

Viva answer: XML Layout Structure defines the hierarchy of ViewGroups and Views used to create an Android application's user interface.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for xml layout structure and test the behaviour in the emulator.

186. @+id and @id

Viva answer: @+id creates a new resource ID, whereas @id refers to an ID that has already been declared.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for @+id and @id and test the behaviour in the emulator.

187. Accessing XML Views Using findViewById()

Viva answer: findViewById() retrieves a View from the Activity's loaded XML layout using its resource ID so that it can be accessed in Java code.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for accessing xml views using findviewbyid() and test the behaviour in the emulator.

188. XML vs Java

Viva answer: XML is mainly used to define the user interface and resources, whereas Java is used to implement application logic and handle user interactions.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for xml vs java and test the behaviour in the emulator.

18. Advanced Android Concepts

189. Main Thread / UI Thread

Viva answer: The Main Thread, also called the UI Thread, is responsible for drawing the user interface and processing user interactions in an Android application.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for main thread / ui thread and test the behaviour in the emulator.

190. Application Not Responding (ANR)

Viva answer: ANR occurs when an application blocks the main thread for too long and stops responding to user input.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for application not responding (anr) and test the behaviour in the emulator.

191. RecyclerView

Viva answer: RecyclerView is a flexible and efficient Android UI component used to display large collections of data by reusing item Views.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for recyclerview and test the behaviour in the emulator.

192. ListView vs RecyclerView

Viva answer: ListView is a simpler list component, whereas RecyclerView provides better performance, flexible layouts, item animations and a more structured ViewHolder-based approach.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for listview vs recyclerview and test the behaviour in the emulator.

193. Android Jetpack

Viva answer: Android Jetpack is a collection of libraries, tools and architectural guidance that helps developers build maintainable Android applications.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for android jetpack and test the behaviour in the emulator.

194. Material Design

Viva answer: Material Design is a design system developed by Google that provides guidelines and components for creating consistent and user-friendly interfaces.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for material design and test the behaviour in the emulator.

195. Kotlin vs Java

Viva answer: Kotlin and Java are programming languages used for Android development, but Kotlin offers concise syntax, null-safety features and official first-class support for modern Android development.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for kotlin vs java and test the behaviour in the emulator.

196. Android Application Security

Viva answer: Android Application Security involves protecting application data and functionality through permissions, secure storage, authentication, encryption and safe communication practices.

Explanation / example: This is used when building or maintaining an Android application. In a practical, connect the concept to the relevant Activity, XML resource or Android API. Example: use the appropriate Android class or XML attribute for android application security and test the behaviour in the emulator.

19. Practical programme explanations

Use this pattern when the examiner asks you to explain your program: state the screen purpose, identify the XML controls, explain the Java event listener, describe validation/storage/navigation, and mention the output.

A. Combined Registration + Login + Student Dashboard

RegisterActivity collects name, email, username and password. It validates empty fields and matching passwords, then saves registration values in SharedPreferences. MainActivity reads the saved username and password and checks them when Login is clicked. On success, an explicit Intent opens DashboardActivity. The dashboard demonstrates Spinner, RadioGroup, CheckBox, ListView, ScrollView, saving/loading student details, AlertDialog, menu actions and implicit Intents.

```
// Save registration values
SharedPreferences sp = getSharedPreferences("MyData", MODE_PRIVATE);
sp.edit().putString("username", u)
      .putString("password", p)
      .putBoolean("registered", true).apply();

// Read and validate
String savedUser = sp.getString("username", "");
String savedPass = sp.getString("password", "");
if (u.equals(savedUser) && p.equals(savedPass)) {
    startActivity(new Intent(MainActivity.this, DashboardActivity.class));
}
```

Examiner may ask: Viva points: SharedPreferences is key-value storage; an explicit Intent names the target Activity; validation must happen before saving or navigating. The simple lab example must not be treated as secure production authentication because it stores a password as plain text.

B. Activity Lifecycle demonstration

Override the lifecycle callbacks and print a Log message in each one. Launch the Activity, press Home, return to it and press Back. Observe the callback sequence in Logcat.

```
@Override protected void onCreate(Bundle b) {
    super.onCreate(b);
    setContentView(R.layout.activity_main);
    Log.d("LIFE", "onCreate");
}
@Override protected void onStart() {
    super.onStart();
    Log.d("LIFE", "onStart");
}
@Override protected void onResume() {
    super.onResume();
    Log.d("LIFE", "onResume");
}
```

Examiner may ask: Initial launch normally calls onCreate, onStart and onResume. Moving fully to background calls onPause and onStop. Returning from stopped state calls onRestart, onStart and onResume. Exact callbacks can vary with system transitions.

C. Calculator using LinearLayout / GridLayout

Place two EditText fields for numbers, operation Buttons and a TextView for the result. Read the input with getText().toString(), convert it using Double.parseDouble(), perform the selected operation and display the result with setText(). Validate empty input and handle division by zero.

```
String a = first.getText().toString().trim();
String b = second.getText().toString().trim();
if (a.isEmpty() || b.isEmpty()) {
    result.setText("Enter both numbers");
} else {
    double x = Double.parseDouble(a);
    double y = Double.parseDouble(b);
    result.setText(String.valueOf(x + y));
}
```

Examiner may ask: LinearLayout arranges controls in a row or column. GridLayout arranges controls in rows and columns. A helper method can avoid repeating the calculation code for every operation.

D. Spinner with event handling

Create an array of options, connect it to an ArrayAdapter, set the Adapter on the Spinner, and register an OnItemSelectedListener. Read the selected item from the callback. The callback may run once during initial setup as well as after user selections.

```
String[] items = {"MCA", "MCS", "BCA"};
ArrayAdapter<String> ad = new ArrayAdapter<>(this,
        android.R.layout.simple_spinner_item, items);
ad.setDropDownViewResource(android.R.layout.simple_spinner_dropdown_item);
spinner.setAdapter(ad);
```

Examiner may ask: An Adapter connects the data array to the Spinner. OnItemSelectedListener has two callbacks: onItemSelected and onNothingSelected.

E. ListView with ArrayAdapter

Create an array or ArrayList, create an ArrayAdapter using a built-in or custom row layout, call setAdapter(), and attach setOnItemClickListener() to respond to item selection.

```
String[] names = {"Java", "Android", "SQLite"};
ArrayAdapter<String> ad = new ArrayAdapter<>(this,
        android.R.layout.simple_list_item_1, names);
listView.setAdapter(ad);
listView.setOnItemClickListener((parent, view, position, id) ->
        Toast.makeText(this, names[position], Toast.LENGTH_SHORT).show());
```

Examiner may ask: simple_list_item_1 displays one text line. The position parameter identifies the selected row.

F. GridView with AlertDialog

Use an Adapter to supply grid items. In the item click listener, obtain the selected item and show an AlertDialog containing its name or details. The dialog can have positive and negative buttons.

```
new AlertDialog.Builder(this)
    .setTitle("Selected item")
    .setMessage("Do you want to continue?")
    .setPositiveButton("Yes", (d, w) -> {
        Toast.makeText(this, "Confirmed", Toast.LENGTH_SHORT).show();
    })
    .setNegativeButton("No", (d, w) -> d.dismiss())
    .show();
```

Examiner may ask: GridView presents items in rows and columns. AlertDialog is used for a confirmation or information message.

G. Options Menu

Define menu items in res/menu XML. Inflate the resource in onCreateOptionsMenu(). Handle the selected item in onOptionsItemSelected() by checking its ID.

```
@Override public boolean onCreateOptionsMenu(Menu menu) {
    getMenuInflater().inflate(R.menu.main_menu, menu);
    return true;
}
@Override public boolean onOptionsItemSelected(MenuItem item) {
    if (item.getItemId() == R.id.menuAbout) {
        Toast.makeText(this, "About", Toast.LENGTH_SHORT).show();
        return true;
    }
    return super.onOptionsItemSelected(item);
}
```

Examiner may ask: MenuInflater converts the XML resource into menu items. Return true when the event is handled.

H. Fragments

Create Fragment classes and their XML layouts. Place a FrameLayout container in the Activity. Use FragmentManager and FragmentTransaction to add or replace a Fragment.

```
getSupportFragmentManager()
    .beginTransaction()
    .replace(R.id.container, new FirstFragment())
    .commit();
```

Examiner may ask: FragmentManager manages Fragments. FragmentTransaction performs the change. commit() schedules the transaction to be applied.

I. Image toggle using FrameLayout

Place an ImageView in a FrameLayout and use a Button or click listener to switch between two drawable resources with setImageResource().

```
final boolean[] on = {false};
button.setOnClickListener(v -> {
    on[0] = !on[0];
    image.setImageResource(on[0] ? R.drawable.on : R.drawable.off);
});
```

Examiner may ask: FrameLayout stacks children. ImageView displays the image. setImageResource() changes the displayed drawable.

J. SQLite mini-project

Create a helper class extending SQLiteOpenHelper. Create the table in onCreate(). Use ContentValues and insert() to add records, rawQuery() or query() to read records, update() to modify records and delete() to remove records.

```
public class DBHelper extends SQLiteOpenHelper {
    public DBHelper(Context c) {
        super(c, "CollegeDB", null, 1);
    }
    @Override public void onCreate(SQLiteDatabase db) {
        db.execSQL("CREATE TABLE student (id INTEGER PRIMARY KEY AUTOINCREMENT, name TEXT, roll INTEGER, course TEXT)");
    }
    @Override public void onUpgrade(SQLiteDatabase db, int oldV, int newV) {
        db.execSQL("DROP TABLE IF EXISTS student");
        onCreate(db);
    }
}
```

Examiner may ask: Use selection arguments such as roll=? instead of concatenating user input into SQL. Close Cursor objects after reading. In a real app, design onUpgrade() to preserve and migrate data rather than blindly dropping the table.

20. SQLite INSERT, SELECT, UPDATE and DELETE

SQLite is an embedded relational database. In Android, SQLiteOpenHelper manages database creation and version upgrades, while SQLiteDatabase performs database operations.

Create table

```
CREATE TABLE student (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT,
    roll INTEGER,
    course TEXT
);
```

INSERT — add a record

```
INSERT INTO student(name, roll, course)
VALUES ('Akhil', 13, 'MCS');

// Android Java
ContentValues cv = new ContentValues();
cv.put("name", "Akhil");
cv.put("roll", 13);
cv.put("course", "MCS");
long newRowId = db.insert("student", null, cv);
```

SELECT — retrieve records

```
SELECT * FROM student;
SELECT * FROM student WHERE roll = 13;

// Android Java
Cursor c = db.rawQuery("SELECT * FROM student", null);
try {
    if (c.moveToFirst()) {
        do {
            String name = c.getString(c.getColumnIndexOrThrow("name"));
            int roll = c.getInt(c.getColumnIndexOrThrow("roll"));
        } while (c.moveToNext());
    }
} finally {
    c.close();
}
```

UPDATE — modify a record

```
UPDATE student SET course = 'MCA' WHERE roll = 13;

// Android Java
ContentValues cv = new ContentValues();
cv.put("course", "MCA");
int rowsUpdated = db.update(
    "student", cv, "roll=?", new String[]{"13"}
);
```

DELETE — remove a record

```
DELETE FROM student WHERE roll = 13;

// Android Java
int rowsDeleted = db.delete(
    "student", "roll=?", new String[]{"13"}
);
```

Critical viva point: never omit WHERE unintentionally. UPDATE without WHERE changes every row; DELETE without WHERE removes every row but leaves the table itself. DROP TABLE removes the table structure and its data.

SQLiteOpenHelper lifecycle

onCreate() is called when the database is first created. onUpgrade() is called when the database version changes. getWritableDatabase() opens a database for reading and writing; getReadableDatabase() opens it for reading and may return a read-only database if writing is unavailable. ContentValues carries column-value pairs. Cursor points to query-result rows.

21. Important differences for viva

Concept A	Concept B	Main difference / example
Explicit Intent	Names the exact target component.	Open DashboardActivity from MainActivity.
Implicit Intent	Describes an action; Android resolves a suitable component.	Open dialler or share text.
SharedPreferences	Small key-value data.	Store simple settings or a demo login flag.
SQLite	Structured records in tables.	Store multiple student records.
Activity	A full Android application component/screen.	LoginActivity.
Fragment	Reusable portion of an Activity UI.	A fragment displayed inside a FrameLayout.
LinearLayout	Arranges children vertically or horizontally.	Vertical form.
RelativeLayout	Positions views relative to parent or other views.	Place a button below a TextView.
FrameLayout	Stacks children in a frame.	Fragment container or image overlay.
GridLayout	Arranges children in rows and columns.	Calculator keypad.
Toast	Brief, non-blocking feedback.	Saved successfully.
AlertDialog	Dialog for information or confirmation.	Confirm delete?
Options Menu	Activity-level actions.	About, Settings, Logout.
Context Menu	Actions for a selected View/item, often on long press.	Edit or delete selected row.
Popup Menu	Small menu anchored to a View.	More actions button.
apply()	Persists changes asynchronously; no result returned.	Common SharedPreferences save.
commit()	Persists synchronously and returns Boolean.	When immediate result is required.
dp	Density-independent layout unit.	Margins and dimensions.
sp	Scalable text unit.	Text size.
DELETE	Removes rows; table remains.	DELETE FROM student WHERE roll=13.
DROP	Removes the table structure and its data.	DROP TABLE student.
ListView	Traditional list widget.	Simple vertical list.
RecyclerView	Flexible, reusable item container with ViewHolder pattern.	Large or complex lists.
Overloading	Same method name, different parameter list.	add(int,int) and add(double,double).
Overriding	Subclass redefines inherited method implementation.	Subclass overrides onCreate().
Padding	Space inside a View boundary.	Text inset from button edge.
Margin	Space outside a View boundary.	Gap between two buttons.

22. Common examiner follow-up questions

Q. Why do we use setContentView()?

A. It loads the XML layout for the Activity so the Activity can display and access its Views.

Q. What is R.id.button?

A. It is the generated resource identifier for the View whose ID is button.

Q. Why do we use findViewById()?

A. It connects a Java variable to a View declared in the loaded XML layout.

Q. What is a Toast?

A. It is a short message used to give temporary feedback.

Q. What is an Adapter?

A. It connects data to a widget that displays a collection of items.

Q. What is an ArrayList?

A. It is a resizable collection that can store objects; unlike an array, its size can change.

Q. Why use trim()?

A. It removes leading and trailing whitespace before validation.

Q. Why use isEmpty()?

A. It checks whether a String contains zero characters.

Q. Why use equals() for passwords?

A. It compares the contents of two String objects.

Q. What does finish() do?

A. It closes the current Activity.

Q. What is MODE_PRIVATE?

A. It makes the SharedPreferences file private to the application by default.

Q. What is a Cursor?

A. It lets the program move through and read rows returned by a database query.

Q. What does moveToFirst() return?

A. It returns true if the Cursor moved to a first row; otherwise false.

Q. What is a primary key?

A. A field that uniquely identifies each row in a table.

Q. What is AUTOINCREMENT?

A. It asks SQLite to generate a new integer key value for inserted rows in the relevant primary-key pattern.

Q. What is SQL injection?

A. It is an attack where untrusted input changes the meaning of an SQL statement; parameterized queries help prevent it.

Q. Why should we close a Cursor?

A. To release resources and avoid leaks.

Q. What is an Intent Filter?

A. A manifest declaration describing the implicit Intent actions, categories and data a component can handle.

Q. What is Log.d()?

A. It writes a debug-level message to Logcat.

Q. What is an ANR?

A. Application Not Responding; it can happen when the main thread is blocked for too long.

Q. Can multiple CheckBoxes be selected?

A. Yes, each CheckBox is independent.

Q. Can multiple RadioButtons in one RadioGroup be selected?

A. Normally no; the RadioGroup enforces one selection at a time.

Q. Why does Spinner use an Adapter?

A. The Adapter supplies the choices that the Spinner displays.

Q. What does notifyDataSetChanged() do?

A. It tells an attached Adapter that its underlying data has changed and the displayed data should be refreshed.

Q. What does commit() return?

A. A Boolean indicating whether the preference changes were successfully persisted.

Q. What does apply() return?

A. It does not return a result; persistence happens asynchronously.

Q. What is the difference between a service and an activity?

A. An Activity provides a user interface; a Service performs work without a normal UI.

Q. What is the difference between a normal and dangerous permission?

A. Dangerous permissions access sensitive data or features and require runtime user approval.

Q. Why use dp for dimensions and sp for text?

A. They adapt better to screen density and user font scaling.

Q. What should you do if the examiner asks about a program you did not implement?

A. Explain the concept honestly, describe the basic workflow and avoid claiming that your combined app implements it.

23. Final practical checklist

Combined application

■ Registration with validation

■ Login with validation

■ SharedPreferences save/read

■ Explicit Activity navigation

■ Implicit dialler/share Intents

■ Activity lifecycle Logcat

■ CheckBox and RadioGroup

■ Spinner and event handling

■ ListView with ArrayAdapter

■ ScrollView

■ AlertDialog

■ Options Menu

Separate practicals to practise

■ LinearLayout calculator

■ GridLayout calculator

■ RelativeLayout page

■ FrameLayout image toggle

■ GridView with AlertDialog

■ Context Menu and Popup Menu

■ Fragments and Fragment navigation

■ Navigation Drawer

■ SQLite INSERT and SELECT

■ SQLite UPDATE and DELETE

■ SQLite mini-project

■ Themes and styles

■ DatePicker and TimePicker

■ MediaPlayer

■ Custom Adapter

Source note: This guide is based on the uploaded MAD.pdf lecture slides and the uploaded 196-topic MAD viva outline. The lecture slides provide supporting diagrams and material (for example, mobile app types and Android architecture on pages 2–4, Android Studio/project setup on pages 5–10, lifecycle on pages 12–13, and UI/display units on pages 14–18). The guide expands the listed syllabus into study explanations and practical examples.

Reminder: This guide is a revision aid, not a guarantee that every examiner question will be covered. Practise writing and running the practical programs yourself, especially SQLite and the separate UI/navigation programs.